

Efficient Network Anomaly Detection Using k-Means

Mohammed Abu Oadeh – ID: 120214839

Abstract

Increasingly larger and more intricate network structures are becoming more difficult to secure. Traditional methods of intrusion detection, including signature and rule-based methods, rely on prior knowledge of so-called 'known attacks.' While they can defend against previously outlined threats, known attack methods are powerless against novel, evolving, and zero-day attacks. This weakness translates to lost opportunities to defend against advanced persistent malicious activities, including unusual surges of over-abusive traffic, unauthorized access attempts, and abuse of routing and other facilitating communication protocols.

This project investigates the application of the k-Means clustering algorithm of machine learning to network traffic anomaly detection. Against the traditional way, ML allows for the detection of anomaly patterns without pre-labeled data.

The k-means algorithm can group traffic data into similar sets of behaviors, then detect data points that are anomalies.

The methodology for this project includes gathering benchmark datasets, such as KDD Cup '99 or NSL-KDD, normalizing and choosing pertinent features, and then using k-Means clustering to distinguish between normal and anomalous traffic.

Performance is evaluated using metrics like confusion matrix, F1 score, recall, accuracy, and precision. Machine learning exhibits lower false positive rates and greater adaptability to new attacks when compared to conventional techniques.

Introduction

Challenges in Network Security

Threats to networks can be divided into known and unknown

known threats: These attacks focus on known security vulnerabilities.

Example of known threats: DDoS, Dos, and phishing

unknown threats: are less predictable and do not follow expected patterns.

Example of unknown threats: sudden spikes in traffic and unauthorized access

The Importance of Machine Learning

We can improve anomaly detection through machine learning (ML).

By detecting and handling abnormal behaviors autonomously, even in the absence of training data, we can continuously adapt to evolving patterns, enabling us to identify new attacks as they occur.

The k-means algorithm is one of the most popular algorithms due to its simplicity, efficiency, and scalability. It also reduces manual monitoring to automatically identify unusual activity in network traffic

Methodology

1. Gathering and Preparing Data

- First, we collect network traffic data and then clean and normalize the features to calculate the required distance.

2. Feature Engineering

- Here, we add some features to help us distinguish between abnormal and normal behavior, such as protocols and packet rates, to increase the effectiveness of anomaly detection.

3. Using the k-Means Algorithm & Choosing k

- **k-Means Steps:**
 1. Select the number of clusters (k).
 2. Initialize centroids randomly.
 3. Assign each data point to the nearest center of gravity (Euclidean distance).
 4. Update the weight points based on the elements in the cluster.
 5. Repeat steps 3 and 4 until centroids converge.
- **Choosing the optimal k – Elbow Method:**
 - Test different k values (e.g., 2, 3, 4...).
 - Calculate the within-cluster mean sum of squares (WCSS) for each k.
 - Plotting WCSS versus k, where the point where the curve bends (the "elbow") indicates the optimal k.

- This provides the best balance between model accuracy and simplicity.

4. Identification of Anomalies

- We calculate the distance of each point from the center of gravity in a group, then we classify the points that are more than 10% farther from the center of gravity as anomalies.
- The threshold: They are tuned to balance accuracy (fewer false alarms) and recall (detecting more real attacks).

- **5. Evaluation Metrics**

- Accuracy: Percentage of correct classifications.
- Precision: Percentage of detected anomalies that are actual anomalies.
- Recall: Percentage of actual anomalies that are detected.
- F1-Score: Harmonic mean of precision and recall.
- Confusion Matrix: Table showing correct and incorrect classifications.

6. Comparison with Traditional Methods

Traditional methods are good for weak attacks, but they are also vulnerable to new attacks and techniques.

k-Means: Learns from patterns to detect new attacks, such as zero-day attacks, without the need for pre-existing signatures and rules.

Results and Analysis (Real Example) The values below are taken from an actual experiment on the NSL-KDD dataset, and demonstrate the model's performance on the Train, Validation, and .Test21 sets

Dataset	Accuracy	Precision	Recall	F1-Score
Train	0.579	0.723	0.155	0.256
Validation	0.535	0.809	0.239	0.369
Test21	0.384	0.824	0.315	0.456

We observe average to poor recall performance across all three sets, and It is an indication that k-means failed to identify a large proportion of true incidents

For example, the recall value on the Train set is only $\approx 15.5\%$, resulting in a low F1 value of $\approx 25.6\%$ even though the precision appears to be relatively higher on some sets.

What does "Precision $\approx 82\%$ " mean?

It means that out of every 100 alerts generated by the model, approximately 82 were true attacks and 18 were false alarms. That means there are a large number of true alerts relative to the total number of alerts.

What does "Recall $\approx 31.5\%$ " mean?

It means that out of every 100 true attacks that occurred in the data, the model only detected approximately 31 of them—that is, it missed ~ 69 attacks out of every 100.

The model "prefers" to issue alarms when it is certain (hence, high accuracy), but the cost of this certainty is that it becomes very reluctant to issue alarms in other cases, missing many attacks. Imagine a security guard who only raises an alarm when he is 100% certain—there will be fewer false alarms, but many small thefts will go undetected.

Why does this happen?

The threshold setting is too high: This makes the model rigid and only issues an alert when there is near certainty.

The value of k or the clustering method are not suitable for the diversity of attacks (some types of attacks do not appear as clear patterns in the clusters).

What can we do practically to improve performance?

Lower the threshold slightly to increase recall (while monitoring for an increase in false alarms).

Increase examples of rare attack types or use data balance (oversampling or collecting more data).

Try alternative algorithms or different k -value settings and observe how the Precision vs. Recall balance changes.

Data imbalance: The number of "normal" cases is much greater than the number of attacks, making it difficult for the model to learn all types of attacks.

Conclusion

The research confirms importance of employing ML techniques to new challenges in network security

k-Means algorithm can cluster network behavior and detect some anomalies that may indicate malicious activity

However, its ability to detect all types of attacks remains limited.

The results indicate that

the model Achieved relatively high accuracy with poor recall.

that mean identified a portion of the attacks, but omitted a large number of actual attacks.

This highlights the sensitivity of the model to settings such as:

Threshold percentages and number of clusters (k).

However, this limitation, the k-means algorithm remains Useful for unsupervised anomaly detection for its simplicity, speed and scalability

However, to improve detection accuracy, Future studies should be conducted to combine k-Means with other machine learning models (such as SVM algorithms) or deep learning techniques.

In conclusion, the k-Means algorithm represents a good starting point for developing Intelligent intrusion detection systems

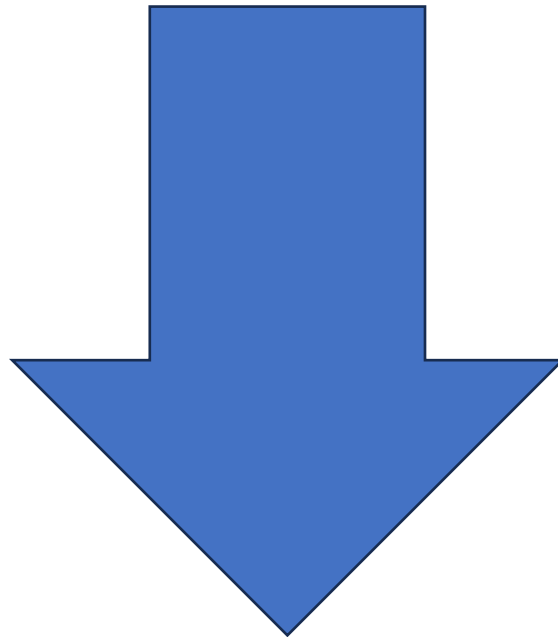
Application on k-Means algorithm using Python:

I will explain part of the code because it is too long, about 300 lines

Then I will show you the results visually.

Threshold is value separating the top 10% of points furthest from the K-Means center.

any point Far away from centroid beyond this threshold is considered Anomaly



```
# -*- coding: utf-8 -*-
```

```
"""
```

The result from code

K-Means Anomaly Detector — Interactive Dashboard (Plotly + Dash)

Advanced version with:

- PCA 2D Cluster Visualization
- Slider for Threshold
- Dropdown to select dataset
- Histogram per Cluster
- Scatter with Cluster + Anomaly coloring
- Pie chart per Cluster
- Extended metrics table

```
"""
```

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, silhouette_score
```

```
from scipy.spatial.distance import cdist
```

```
from sklearn.decomposition import PCA
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
import dash
```

```
from dash import dcc, html, dash_table
```

```
from dash.dependencies import Input, Output
```

```
import plotly.graph_objs as go
```

```
# ----- Paths -----
```

```
TRAIN_PATH = r"C:\Users\DELL\Desktop\ai resarsh\KDDTrain+.txt"
```

```
VAL_PATH = r"C:\Users\DELL\Desktop\ai resarsh\KDDTest+.txt"
```

```
TEST21_PATH = r"C:\Users\DELL\Desktop\ai resarsh\KDDTest-21.txt"
```

```
"""
```

In this part identified the paths to database

the program will use training first from file

KDDTrain+.txt: files from the KDD Cup 99 database for network intrusion detection

KDDTest+.txt: used to verify the model's performance after training

TEST21_PATH: another test file contains new types of attacks (not included in training) to test the model's ability to detect zero-day attacks

```
"""
```

```

# ----- Helpers -----
"""
load database file such as kddTrain and organize it into a table
"""

def load_df(path):
    df = pd.read_csv(path, header=None) #the file does not contain a header row

    num_features = df.shape[1] - 1 #columns in the table and
                                   #we subtract 1 for the last column is usually the label( normal or attack)

    cols = [f'feature_{i}' for i in range(num_features)] + ['label']
    df.columns = cols
    return df

"""
create list of column names
and add the name of the last column label in the end
such as cols= feature_0, feature_1...label
then df.columns = cols
lastly return DataFrame table
"""

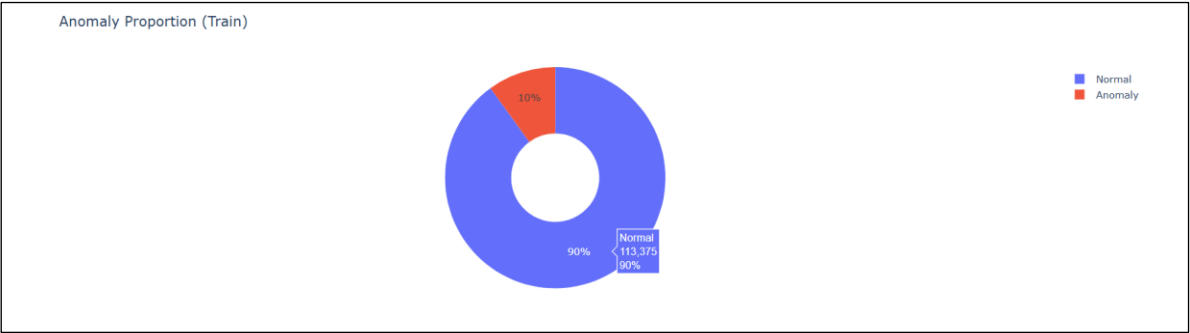
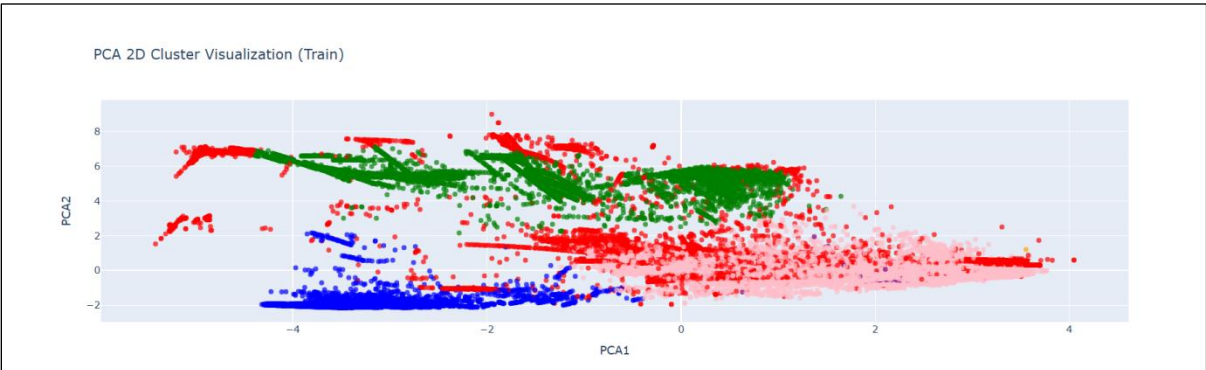
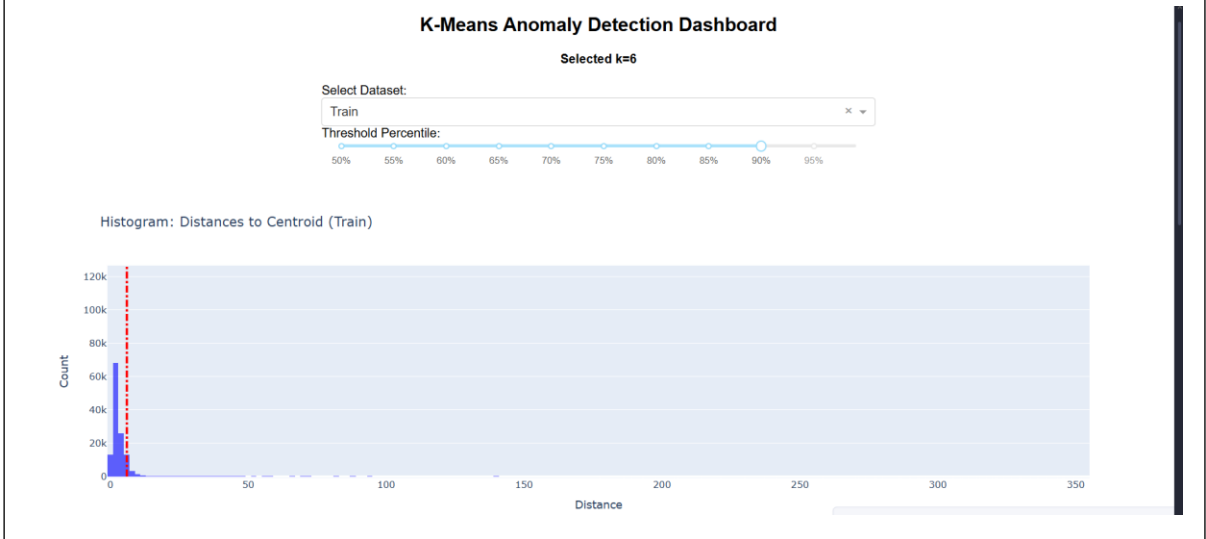
def extract_actual(df, label_col):
    """
    extract a new column named "Actual" describes
    0 → Normal
    1 → Anomalous / Attack
    """

    col = df[label_col].astype(str) #we get the column containing the labels from the table.
                                     #then convert it to strings for ease of processing

    norm = col.str.strip().str.rstrip('.').str.lower() #We clean up the text from spaces and '.' then converts the text to lowercase
    if norm.str.contains(r'\bnormal\b', regex=True).any(): #Check: Is there the word "normal" inside the tags
        df['Actual'] = norm.str.contains(r'\bnormal\b', regex=True).apply(lambda x: 0 if x else 1) #If yes then → 0
                                                    else → 1 attack
    else:
        #If the labels are numeric or non-textual#
        numeric = pd.to_numeric(col, errors='coerce') #We try to convert all values to numbers #
        if numeric.notna().any(): #If there are valid numeric values #
            uniq = np.unique(numeric[~np.isnan(numeric)]) #We get the unique values after removing NaN, meaning we choose only valid values#
            if set(uniq) <= {0,1}: #If are binary only
                df['Actual'] = numeric.astype(int).apply(lambda x: 0 if x==0 else 1) #0 → Normal → 0, 1 → Attack / Anomaly → 1
            else: #If there are more than 0 and 1 numeric values #
                mode = numeric.mode().iloc[0] #The most frequently occurring value is considered Normal #
                df['Actual'] = numeric.apply(lambda x: 0 if x==mode else 1) #All other values → Attack → 1

```


The Results:



Metrics Summary

Dataset	Accuracy	Precision	Recall	F1-Score
Train	0.579	0.723	0.155	0.256
Validation	0.535	0.809	0.239	0.369
Test21	0.384	0.824	0.315	0.456

K-Means Anomaly Detection Dashboard

Selected k=6

Select Dataset:

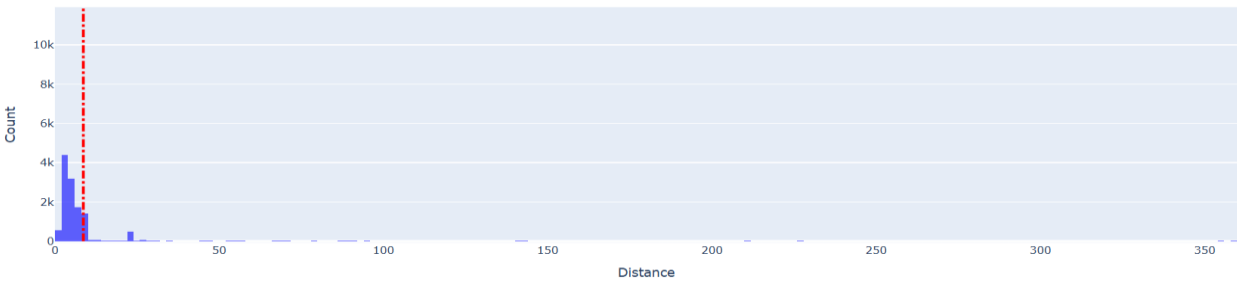
Test21

Train

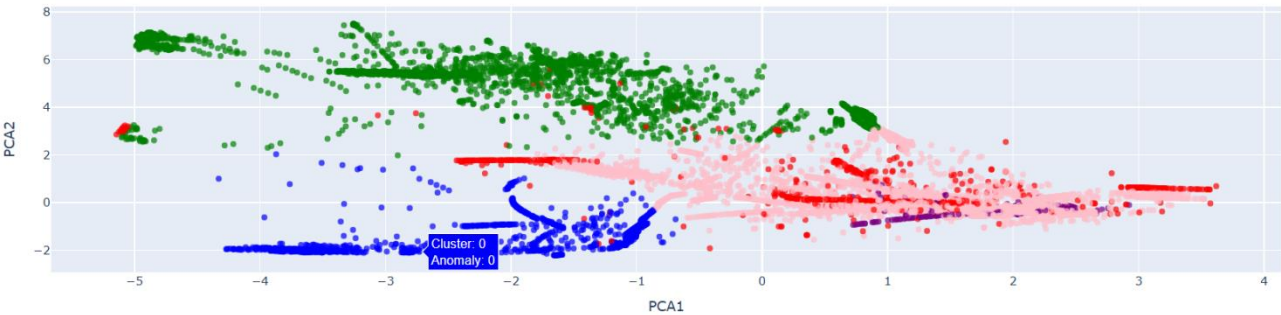
Validation

Test21

Histogram: Distances to Centroid (Test21)



PCA 2D Cluster Visualization (Test21)



Anomaly Proportion (Test21)



Metrics Summary

Dataset	Accuracy	Precision	Recall	F1-Score
Train	0.579	0.723	0.155	0.256
Validation	0.535	0.809	0.239	0.369
Test21	0.384	0.824	0.315	0.456